Software Requirements Specification

MEA-V2G / FreeV2G

Reference Implementation for ISO 15118 EV/EVSE

Ruslee Sutthaweekul

December 24, 2025

Contents

Chapter 1

Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the MEA-V2G (FreeV2G) project. This system serves as a reference implementation for controlling the 8devices WHITE-beet-EI (EVSE) and WHITE-beet-PI (EV) ISO 15118 modules using an Ethernet host control interface (HCI).

1.2 Scope

The MEA-V2G software encompasses:

- Parsing of the protocol used to communicate with the WHITE-beet modules.
- Simulation of EVSE and EV behaviors.
- Handling of charging parameters (Voltage, Current, Power).
- Implementation of basic Control Pilot (CP) and SLAC sequences.
- High-level V2G communication management.
- Integration with OCPP 2.0.1 for backend communication.

1.3 Definitions and Acronyms

EV Electric Vehicle

EVSE Electric Vehicle Supply Equipment (Charging Station)

HCI Host Control Interface

SLAC Signal Level Attenuation Characterization

V2G Vehicle-to-Grid

PNC Plug & Charge

EIM External Identification Means

OCPP Open Charge Point Protocol

CSMS Charging Station Management System

Chapter 2

Overall Description

2.1 Product Perspective

The software interacts directly with the WHITE-beet hardware modules via Ethernet or SPI. It acts as the "Host" controller, managing the logic while the WHITE-beet module handles the low-level PLC (Power Line Communication) and ISO 15118 framing.

2.2 System Functions

The system enables the following core functions:

1. **Control Pilot (CP) implementation**: Detects EV plugin and manages duty cycles.
2. **SLAC**: Automates the matching process between EV and EVSE at the physical layer.
3. **V2G Session Management**: Handles Service Discovery, Parameter Discovery, and Authorization.
4. **Charging Loop**: Monitoring and control of the active energy transfer.
5. **OCPP Communication**: Interface with a CSMS via OCPP 2.0.1 for remote control and status monitoring.

2.3 Use Case Diagram

The following diagram illustrates the primary actors and use cases for the MEA-V2G system.

2.4 System Architecture

The system obeys a modular object-oriented architecture. The core logic handles the state machines for EV and EVSE, while the ‘Whitebeet‘ class abstracts the hardware communication.

2.5 EV State Machine

The EV internal logic follows a distinct state machine to manage the charging session lifecycle, as observed in the implementation.

Init Initial state where the EV controller is started and hardware interfaces (SPI/Ethernet) are initialized.

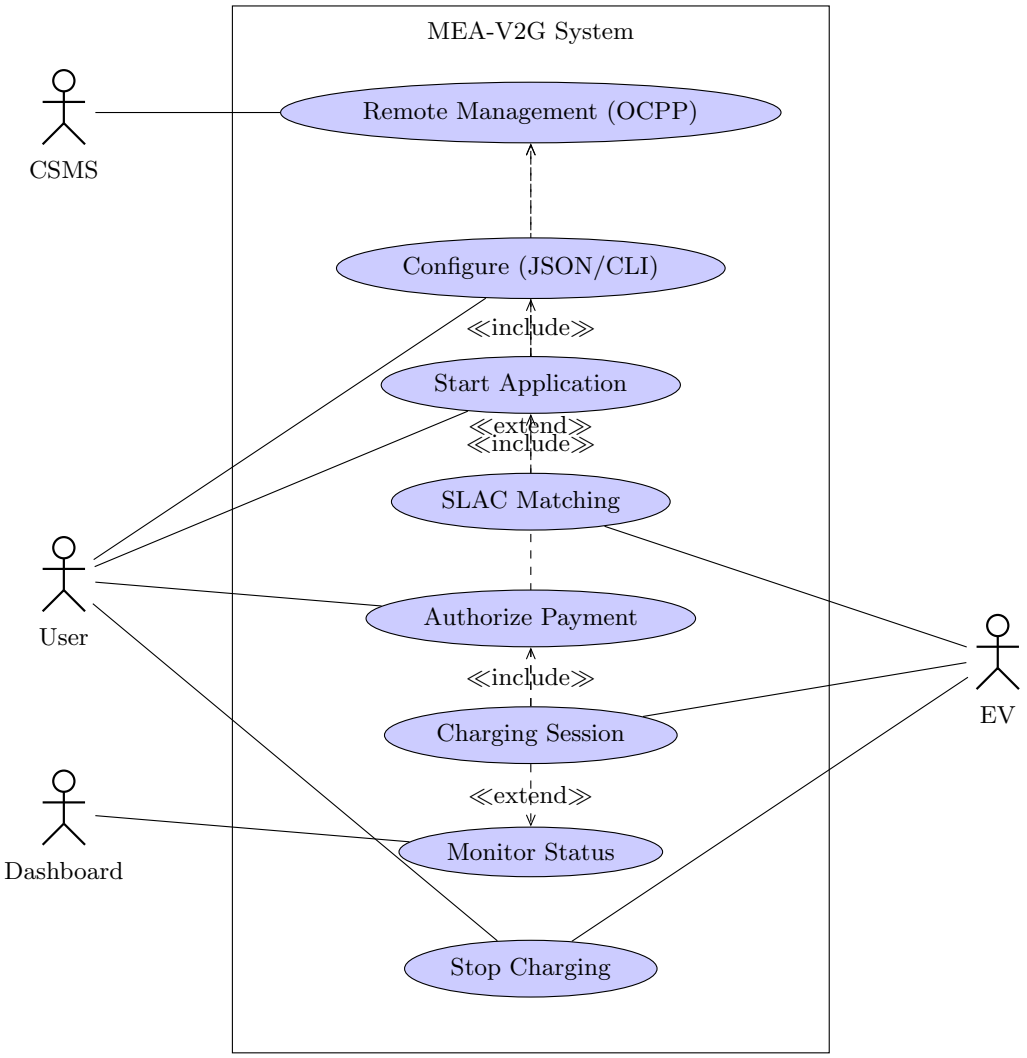

Figure 2.1: MEA-V2G High-Level Use Case Diagram

SessionStarted The V2G session has been established. The EV has successfully matched with the EVSE via SLAC.

CableCheck The EV waits for the EVSE to perform the cable check (isolation test) to ensure safety.

PreCharging Pre-charge phase where the EV matches its battery voltage to the DC bus voltage.

Charging The main state where energy transfer occurs. The EV sends *CurrentDemand* requests in a loop.

ChargingStopped The session is terminated, either normally (target SOC reached) or due to an error.

2.6 EVSE State Machine

The EVSE logic complements the EV, acting as the server component that handles session setup, authorization, and power delivery control.

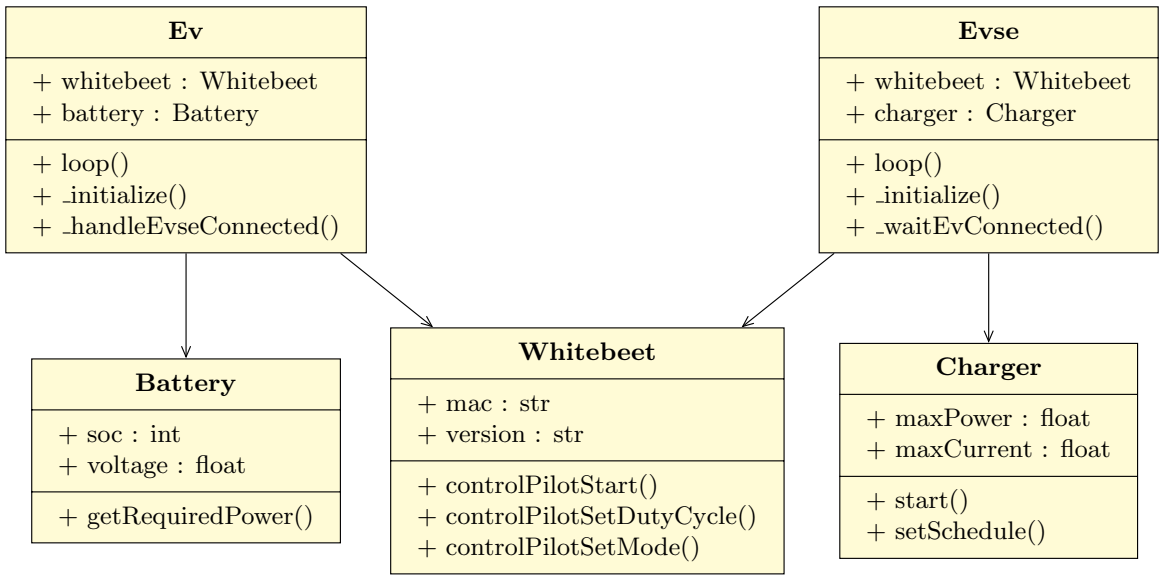

Figure 2.2: MEA-V2G Simplified Class Diagram

Init Initialization of the Whitebeet module services (CP, SLAC, V2G).

Wait The EVSE waits for an EV to physically plug in (Control Pilot State B).

SLAC Signal Level Attenuation Characterization process to pair the EV and EVSE.

Session High-level communication session established; protocol negotiation occurs.

Authorization Verifying if the EV is authorized to charge (via EIM or Plug & Charge).

Params Exchanging charge parameter limits (Maximum Voltage, Current, Power limits).

Cable Performing cable isolation check to ensure no faults exist.

PreCharge Adjusting DC output voltage to match the EV's reported battery voltage.

Charge Active energy transfer loop responding to the EV's current demand.

Stop Session termination and welding detection sequence.

Faulted Critical error state (e.g., GFDI trip, Overcurrent). The EVSE opens contactors and sends a `StatusNotification(Faulted)` to the CSMS via OCPP. It awaits a manual or remote reset.

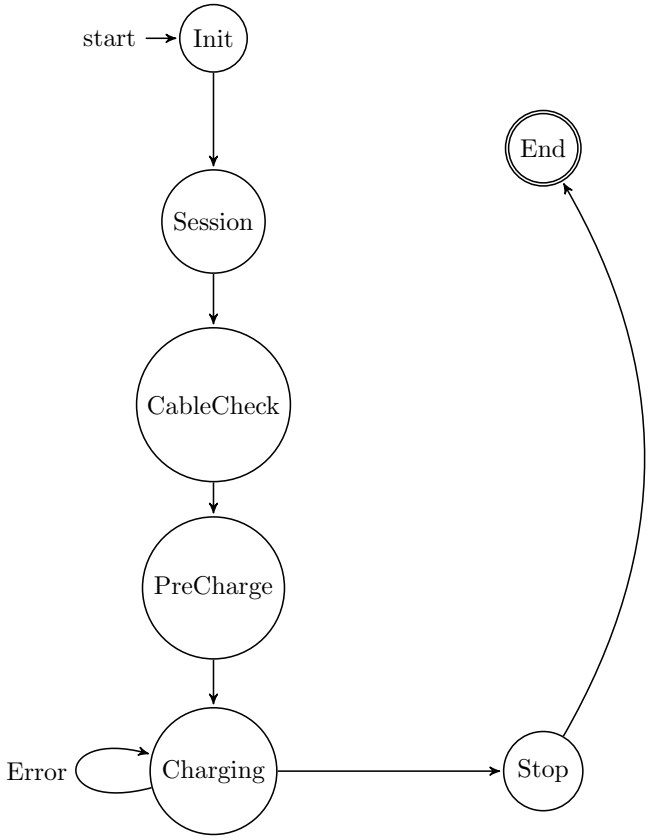

Figure 2.3: EV State Machine

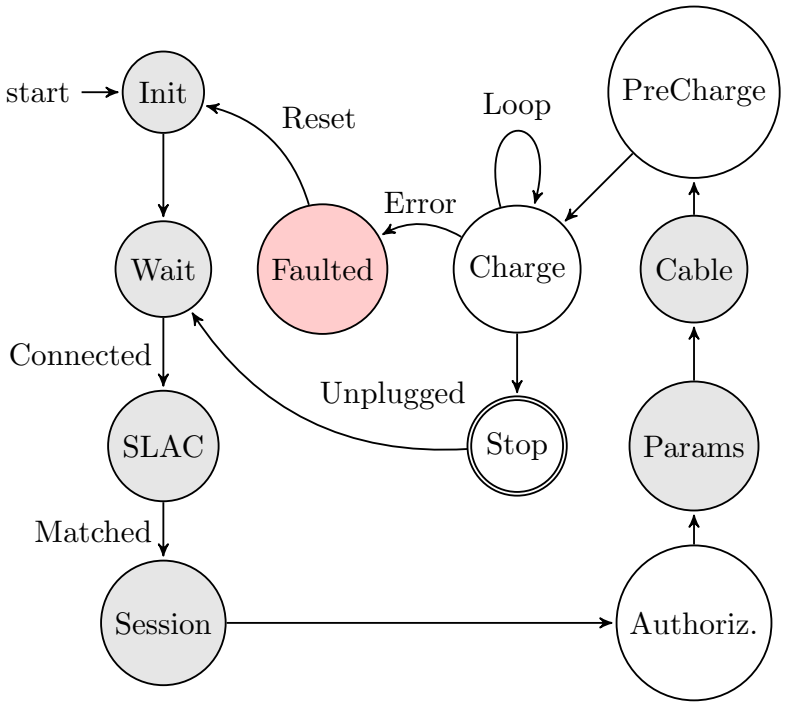

Figure 2.4: EVSE State Machine

2.7 State Synchronization

The charging process relies on the tight synchronization between the EV and EVSE states. Figure 2.5 illustrates the parallel state progression and key synchronization points.

Discovery Control Pilot signal detection (State A → B).

Setup SLAC matching and V2G Session Setup.

Safety Both sides must confirm safety parameters during Cable Check.

Sync Voltage levels must be synchronized during Pre-Charge before the contactors close.

Power The Charging Loop is a lock-step process where the EV requests and the EVSE delivers.

Term Coordinated shutdown to ensure contactors open safely (Welding Detection).

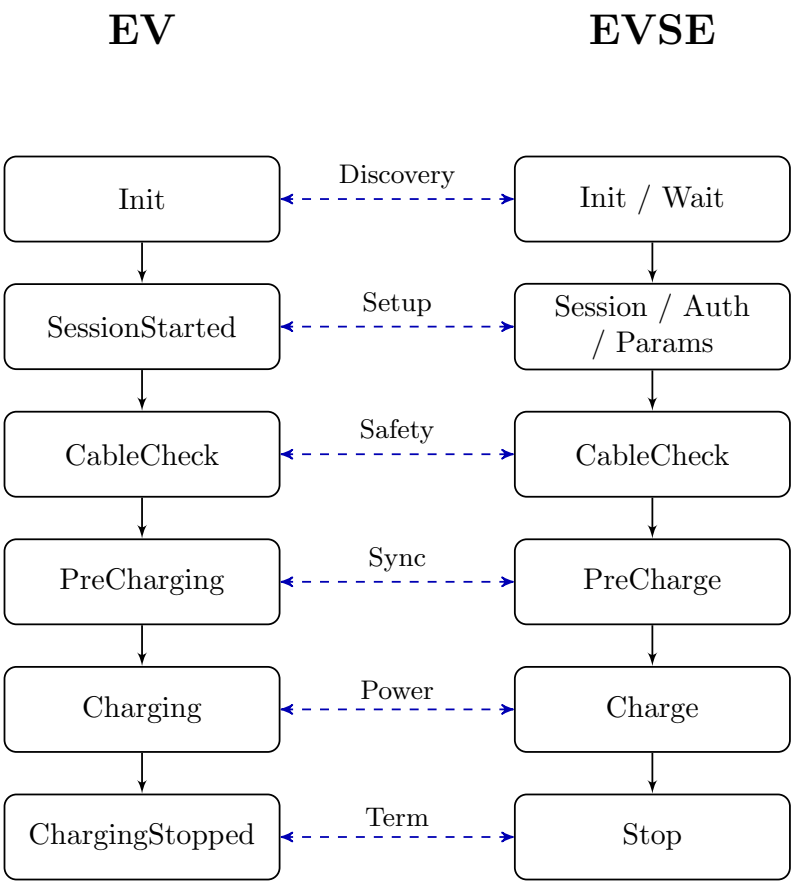

Figure 2.5: EV-EVSE State Synchronization

2.8 Charging Loop Sequence

The following sequence diagram depicts the simplified ISO 15118 charging loop interaction between the EV and EVSE, which is the core logical flow of the application.

2.8.1 CSMS-EVSE-EV Interaction

This diagram focuses on the 3-tier communication flow involving the management system, the charging station, and the vehicle. It illustrates a scenario where a remote command initiates a

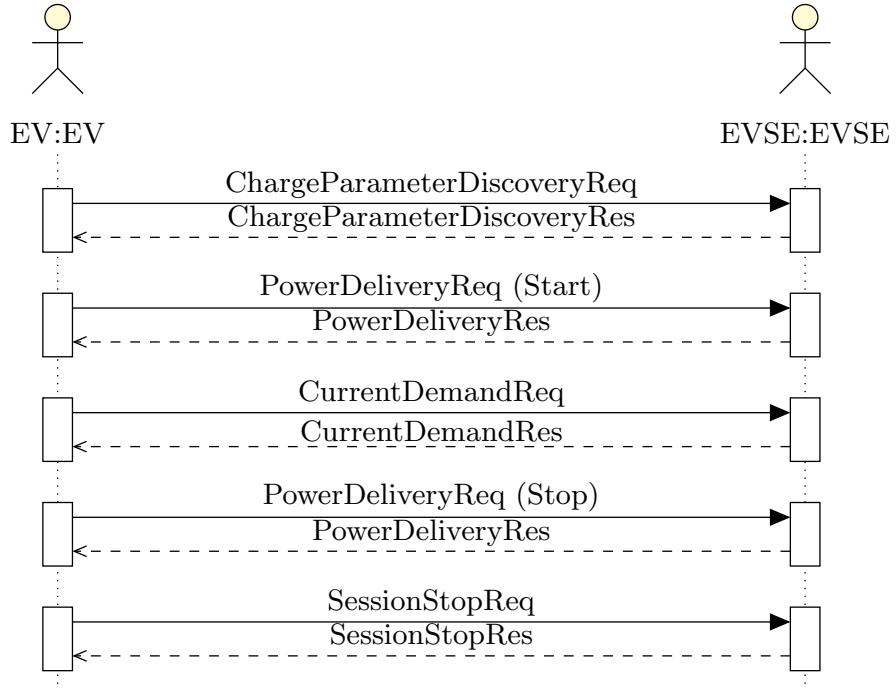

Figure 2.6: ISO 15118 Charging Loop Sequence

session with smart charging limits.

2.9 Core Components

The system comprises the following key software components:

2.9.1 Whitebeet Component

The `Whitebeet` class serves as the Hardware Abstraction Layer (HAL). It manages the low-level communication with the physical ISO 15118 module via Ethernet or SPI.

- **Packet Framing**: Handles the encapsulation of Host Control Interface (HCI) commands.
- **Event Dispatching**: Receives raw data from the hardware and dispatches events (e.g., `EV_CONNECTED`, `V2G_MESSAGE`) to the higher-level logic.
- **Service Management**: Exposes methods to control Control Pilot (CP) state, SLAC, and V2G messaging.

Protocol Structure

The Whitebeet module communicates via a serial interface using a framed protocol. Each frame is structured as follows:

Start	**Mod ID**	**Sub ID**	**Req ID**	**Len (2)**	**Payload**	**CRC**	**End**
0xC0	1 Byte	1 Byte	1 Byte	Big Endian	N Bytes	1 Byte	0xC1

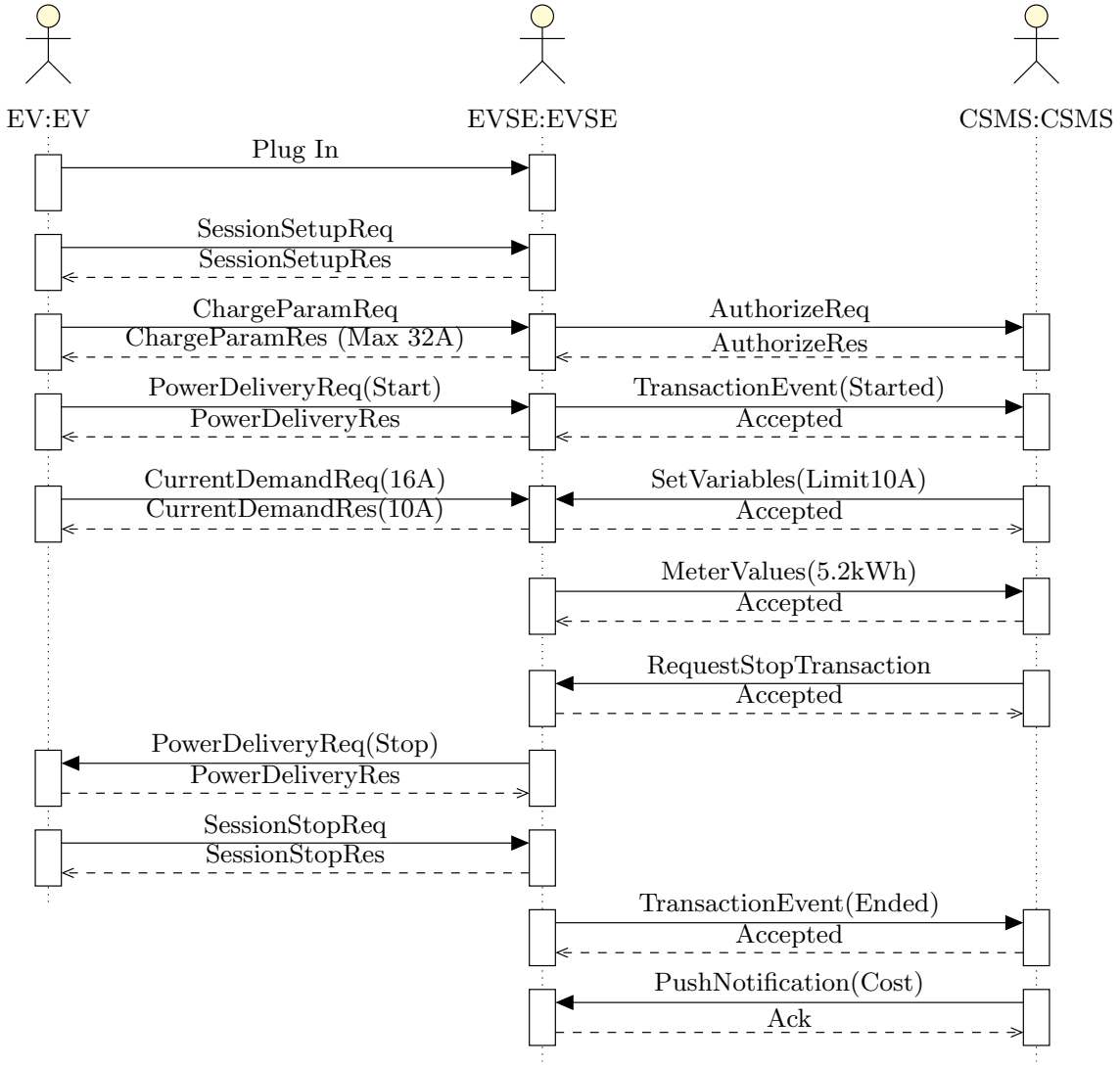

Figure 2.7: 3-Tier (CSMS-EVSE-EV) Sequence Flow

HCI State Transition Mapping

The following table maps the critical EVSE states to the corresponding ISO 15118 V2G messages and the Whitebeet HCI notifications that trigger the transitions.

EVSE State	ISO 15118 Message	HCI Notification	HCI ID
CableCheck	`ChargeParameterDiscoveryReq`	`SessionStarted`	0x80
Authorization	`AuthorizationReq`	`RequestAuthorization`	0x82
PreCharge	`PowerDeliveryReq(Start)`	`PowerDeliveryReq`	0x8E
PreCharge	`PreChargeReq`	`PreChargeStarted`	0x88
Charge	`CurrentDemandReq`	`DCChargeParamsChanged`	0x85
Stop	`PowerDeliveryReq(Stop)`	`RequestStopCharging`	0x8A
Stop	`SessionStopReq`	`SessionStopped`	0x8C
Faulted	(Any Error)	`SessionError`	0x8E

Table 2.1: Whitebeet HCI State Mapping

2.9.2 EV Component

The `EV` class is the central controller for the client side. It orchestrates the charging session from the vehicle's perspective.

- **State Machine**: Implements the ISO 15118 client flow (Session Setup → Charge Parameter Discovery → Power Delivery).
- **Hardware Aggregation**: Owning instance of `Whitebeet` (for comms) and `Battery` (for energy simulation).

2.9.3 EVSE Component

The `EVSE` class is the central controller for the station side. It manages the server-side logic and power delivery authorization.

- **State Machine**: Implements the ISO 15118 server flow, including Service Discovery and Plug & Charge authorization.
- **Hardware Aggregation**: Owning instance of `Whitebeet` (for comms) and `Charger` (for power delivery simulation).
- **Scheduling**: Manages SASchedules based on configuration.

2.9.4 OCPP Component

The system integrates an OCPP 2.0.1 client to enable remote management.

- **OcppInterface**: Adapts the `ocpp` library's `ChargePoint` class to the MEA-V2G architecture. It maps OCPP commands (e.g., `RequestStartTransaction`) to local `Charger` actions.
- **OcppWorker**: A dedicated thread that manages the `asyncio` event loop and WebSocket connection to the CSMS, ensuring non-blocking operation alongside the main application loop.

2.10 Charger Component

The `Charger` class performs a physical simulation of the power electronics, ensuring realistic behavior during the charging session. Its key functions include:

- **Output Ramping**: Instead of instantaneous jumps, the charger simulates the physical "slewing" of voltage and current over time. It uses defined ΔU and ΔI rates to ramp the output toward the target setpoint.
- **Limit Enforcement**: It strictly enforces hard safety limits for the system (e.g., `evse_max_current`, `evse_max_power`), protecting the modeled hardware.
- **Setpoint Control**: The EV provides dynamic target values (*Target Voltage*, *Target Current*), which the charger attempts to follow within its safety constraints.
- **Fault Detection**: The class includes methods like `isVoltageLimitExceeded()` and `isPowerLimitExceed` to trigger immediate session termination if operating conditions are violated.

The `Charger` class implements the `ChargerInterface` abstract base class. This design pattern allows the simulation module to be easily swapped with a real hardware driver, such as the `CanCharger` class, which communicates with physical power electronics via CAN bus using the `python-can` library, while maintaining a consistent API for the `EVSE` logic.

2.10.1 Implementing a Custom Charger

To integrate specific power electronics hardware, developers must create a subclass of `ChargerInterface`. This ensures the `EVSE` logic remains decoupled from the physical layer.

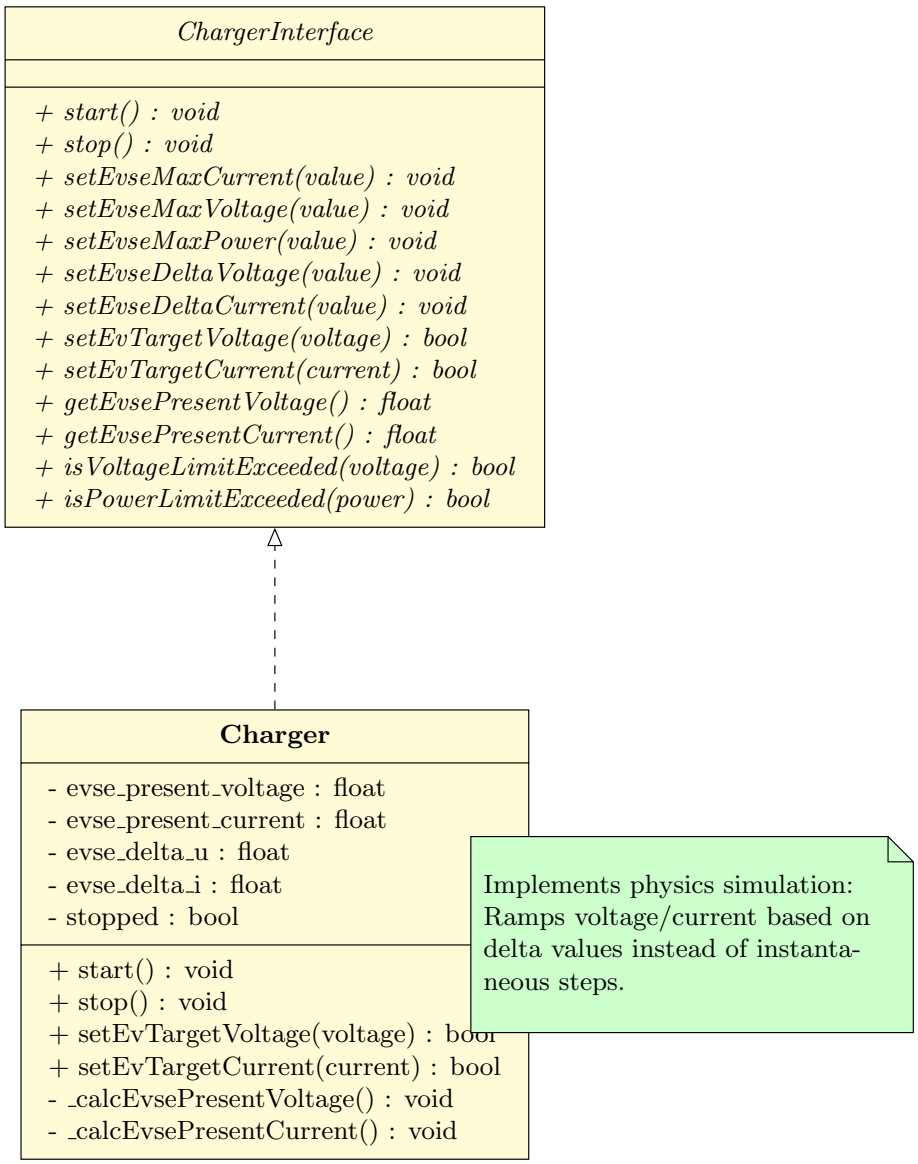

Figure 2.8: Charger Interface and Implementation

A custom implementation must override the abstract methods to map high-level commands to hardware-specific protocols. The following sections detail the requirements for each group of methods.

Lifecycle Management

These methods control the high-level operational state of the power electronics.

`start()` Called when the EVSE initializes or prepares for a session. It should initialize hardware communication (e.g., open CAN sockets). **Note**: This method should **not** automatically close the output contactors; that is handled by the state machine after safe voltage matching.

`stop()` **Critical Safety Method**. Called to terminate a session or during error conditions. It must immediately disable power output, open contactors, and return the hardware to a safe idle state (0V/0A).

Control Loop Methods

These methods are called cyclically (typically every 10-100ms) during the active charging phase.

`setEvTargetVoltage(val)` Receives the EV's requested voltage setpoint (in Volts). The implementation must transmit this request to the power supply unit (PSU).

`setEvTargetCurrent(val)` Receives the EV's requested current setpoint (in Amps). The implementation must ensure the PSU imposes this current limit.

Return Value: These methods must return `True` if the command was successfully queued or sent to the hardware, and `False` if the request triggers a local hardware fault or limit violation.

Telemetry Methods

The EVSE state machine relies on accurate feedback to perform safety checks (e.g., Welding Detection) and process steps (e.g., Pre-Charge).

`getEvsePresentVoltage()` Must return the *measured* voltage at the DC output terminals. Do not return the internal setpoint; the actual value is required to verify contactor states.

`getEvsePresentCurrent()` Must return the *measured* current flowing to the EV.

Configuration Accessors

The interface includes getters and setters for electrical limits (e.g., `setEvseMaxCurrent`, `getEvseMaxPower`).

- For a **Simulation** (like `Charger.py`), these methods simply store and retrieve variables to emulate different hardware capabilities.
- For **Physical Hardware**, the setters (e.g., 'setEvseMaxCurrent') should theoretically configure the hardware's internal safety latches if supported. The getters must return the valid limits to be advertised to the EV during the *Charge Parameter Discovery* phase.

2.10.2 TRIO-HP Charger Protocol

This section details the specific hardware and communication protocol for the TRIO-HP/3AC/1KDC/20KW/BI power module, which is supported by the system via a CAN bus interface.

Hardware Description

- **Brand**: PHOENIX CONTACT
- **Model**: TRIO-HP/3AC/1KDC/20KW/BI
- **Function**: Bidirectional DC/AC and AC/DC converter.
- **Communication**: CAN 2.0B Extended Frame (29-bit Identifier).
- **Baud Rate**: 125 kbps.
- **Termination**: 120 Ω resistors required.

Protocol Structure

The protocol uses a 29-bit extended identifier structured as follows:

Error (3 bits)	Device No. (4 bits)	Command (6 bits)	Target (8 bits)	Source (8 bits)
28-26	25-22	21-16	15-8	7-0

- **Error Code**: 0x00 = Normal.
- **Device No**: 0x0A for individual modules, 0x0B for groups.
- **Target/Source Address**: Unique module ID; Controller default is 0xF0.

Key Commands

The system utilizes the following CAN commands to control the charger:

Description	Cmd ID	Data Bytes (Hex)	Notes
Read DC Voltage	0x23	10 01 00 00 ...	Response contains voltage in mV.
Read DC Current	0x23	10 02 00 00 ...	Response contains current in mA.
Set DC Voltage	0x24	10 01 00 00 ...	Sets output voltage target (mV).
Set DC Current	0x24	10 02 00 00 ...	Sets output current limit (mA).
Enable Operation	0x24	11 10 00 00 ...	Byte 7: 0xA0 (On), 0xA1 (Off).

Protocol Integration

The following table maps the abstract EVSE states and OCPP events to specific CAN protocol actions, ensuring the hardware behaves correctly during the session lifecycle.

EVSE State	Event / Trigger	System Action	CAN Command Sequence
PreCharge	Session Setup	Match EV battery voltage to prevent inrush current.	`Set DC Voltage` (Target) `Read DC Voltage` (Verify) `Enable Operation` (0xA0)
Charge	`SetChargingProfile` (OCPP)	Apply new maximum current limit from CSMS.	`Set DC Current` (Profile Limit)
Charge	`CurrentDemand` (EV)	Follow EV's dynamic current request.	`Set DC Current` (EV Target)
Stop	Session Check / User Stop	Safe shutdown sequence.	`Set DC Current` (0 A) `Read DC Current` (Verify 0A) `Disable Operation` (0xA1)

CanCharger Implementation

The `CanCharger` class provides the concrete implementation for the TRIO-HP protocol. It utilizes the `python-can` library to handle low-level bus arbitration and message framing.

- **Bus Abstraction**: Supports multiple interfaces (SocketCAN, Kvaser, Virtual) via the library's abstraction layer.

- **Threaded Listening**: A continuous background thread monitors the bus for broadcast updates (e.g., current system voltage) to keep the internal state synchronized with the hardware.
- **Error Handling**: Automatically detects connection failures or missing libraries, falling back to safe states or raising initialization errors.

2.11 Battery Component

The `Battery` class provides a logical model of the EV's energy storage, managing State of Charge (SOC) and physical limits.

- **Energy Tracking**: Maintains the battery capacity (Wh) and current energy level. SOC is dynamically calculated as $\frac{Current\ Level}{Capacity} \times 100$.
- **Charging Process**: The `tickSimulation()` method accumulates energy over time intervals based on the instantaneous input voltage and current ($E = V \times I \times \Delta t$).
- **Configurable Limits**: Supports defining maximum Voltage, Current, and Power constraints specific to the chosen Energy Transfer Mode (AC vs. DC).
- **Status Reporting**: Continuously updates the SOC and charging status ('is_full', 'is_charging'), which are reported back to the EVSE via ISO 15118 messages.

2.12 Default Charging Routine

In the absence of an external configuration file, the system defaults to a hardcoded "Simple Charging Routine" defined in `Application.py`. This routine ensures the EVSE behaves predictably during initial testing or standalone operation.

2.12.1 Default Schedule

The default charging profile is a time-dependent power curve that simulates a decreasing power availability (e.g., grid balancing or solar curve):

- **0 to 30 minutes**: Maximum Power 25,000 W.
- **30 to 60 minutes**: Maximum Power reduced to 18,750 W.
- **Over 60 minutes**: Maximum Power further reduced to 12,500 W.

2.12.2 Physics Simulation Parameters

To simulate realistic hardware behavior, the `Charger` class uses specific delta parameters to ramp voltage and current rather than applying instantaneous steps:

- **Delta Voltage**: 0.5 V per tick.
- **Delta Current**: 0.05 A per tick.
- **Max Limits**: 400 V / 100 A / 25 kW.

2.13 Project File Structure

The following table summarizes the key files in the repository and their specific purposes within the architecture:

File Name	Description
Application.py	The main entry point. Parses command-line arguments and initializes the `EV` or `EVSE` instance based on the selected role.
Ev.py	Implements the Electric Vehicle (client) logic, including the ISO 15118 state machine and session management.
Evse.py	Implements the Charging Station (server) logic, managing service discovery, authorization, and power delivery.
Whitebeet.py	The Hardware Abstraction Layer (HAL). Handles low-level communication with the Codico Whitebeet module.
Charger.py	A physics simulation class for the EVSE's power electronics. Simulates voltage/current ramping and enforces safety limits.
ChargerInterface.py	An abstract base class defining the standard API for charger implementations, enabling pluggable drivers (Simulation vs. CAN).
Battery.py	A logical model of the EV battery, tracking State of Charge (SOC) and energy capacity.
EthernetAdapter.py	Driver for communicating with the Whitebeet module via a network socket.
SpiAdapter.py	Driver for communicating with the Whitebeet module via the SPI bus (e.g., on Raspberry Pi).
FramingInterface.py	Defines the packet structure and framing rules for the Host Control Interface (HCI) protocol.
api_server.py	A lightweight Flask server that exposes internal state (like SOC, current power) via a REST API for monitoring.
gui_launcher.py	A graphical utility to easily launch `Application.py` without using the command line.
ev.json / evse.json	Configuration files defining capabilities, limits, and schedules for the respective roles.

Chapter 3

Specific Requirements

3.1 External Interfaces

3.1.1 Communication Interfaces

- **Ethernet**: Main control interface for sending HCI commands and receiving notifications.
- **SPI**: Alternative control interface (e.g., for Raspberry Pi integration).
- **OCPP Interface**: WebSocket connection (JSON over WebSockets) to a central management system (CSMS).

3.1.2 User Interfaces

- **CLI (Command Line Interface)**: Primary mode of operation. Outputs session logs and accepts interactive input (e.g., authorization "yes/no").
- **GUI**: A graphical launcher (‘gui_launcher.py‘) provided for configuration and startup.
- **API**: A REST API (default port 5000) for real-time status monitoring (e.g., for Grafana dashboards).

3.2 Functional Requirements

3.2.1 FR-01: Protocol Negotiation

The system shall support negotiation of ISO 15118 protocol versions.

- The EVSE shall advertise supported protocols.
- The EV shall select a mutually supported protocol (e.g., DIN SPEC 70121 or ISO 15118-2).

3.2.2 FR-01-A: OCPP Version Selection

The system shall allow the selection of the OCPP protocol version at startup.

- Supported versions: OCPP 1.6, OCPP 2.0.1, and OCPP 2.1 (experimental).
- The version shall be configurable via command-line arguments.

3.2.3 FR-02: Authorization

The system shall support multiple authorization modes:

- **EIM (External Identification Means)**: Manual authorization via console input.
- **PnC (Plug & Charge)**: Automated certificate-based authorization.

3.2.4 FR-03: Charging Control

During the charging loop, the system shall:

- Exchange charging parameters (SOC, Target Voltage, Target Current).
- Terminate the session upon "Stop Charging" request from either side.

3.2.5 FR-04: OCPP Integration

The system shall implement OCPP client features for the selected version (1.6, 2.0.1, or 2.1):

- Connect to a CSMS via WebSocket.
- Send version-specific `BootNotification` and `Heartbeat` messages.
- Accept remote commands appropriate for the version (e.g., `RemoteStartTransaction` for 1.6, `RequestStartTransaction` for 2.0.1+).
- Support configuration/variable management (`ChangeConfiguration` for 1.6, `SetVariables` for 2.0.1+).

3.3 Communication Flow

3.3.1 OCPP 1.6 Sequence Flow

The interaction in OCPP 1.6 is similar but uses different message names and structures. The following diagram illustrates the flow for OCPP 1.6 (JSON).

3.3.2 OCPP 2.0.1 Sequence Flow

The following diagram illustrates the interaction between the EVSE (Charge Point) and the CSMS (Charging Station Management System) via OCPP 2.0.1.

3.3.3 Key Differences: OCPP 1.6 vs 2.0.1

The following table highlights the primary differences in message terminology and functionality between the two supported versions.

Feature	**OCPP 1.6**	**OCPP 2.0.1**
Remote Start	`RemoteStartTransaction`	`RequestStartTransaction`
Remote Stop	`RemoteStopTransaction`	`RequestStopTransaction`
Configuration	`ChangeConfiguration`	`SetVariables`
Status	`StatusNotification`	`StatusNotification` (Expanded states)
Transaction	`StartTransaction`, `StopTransaction`	`TransactionEvent` (Started/Updated/Ended)

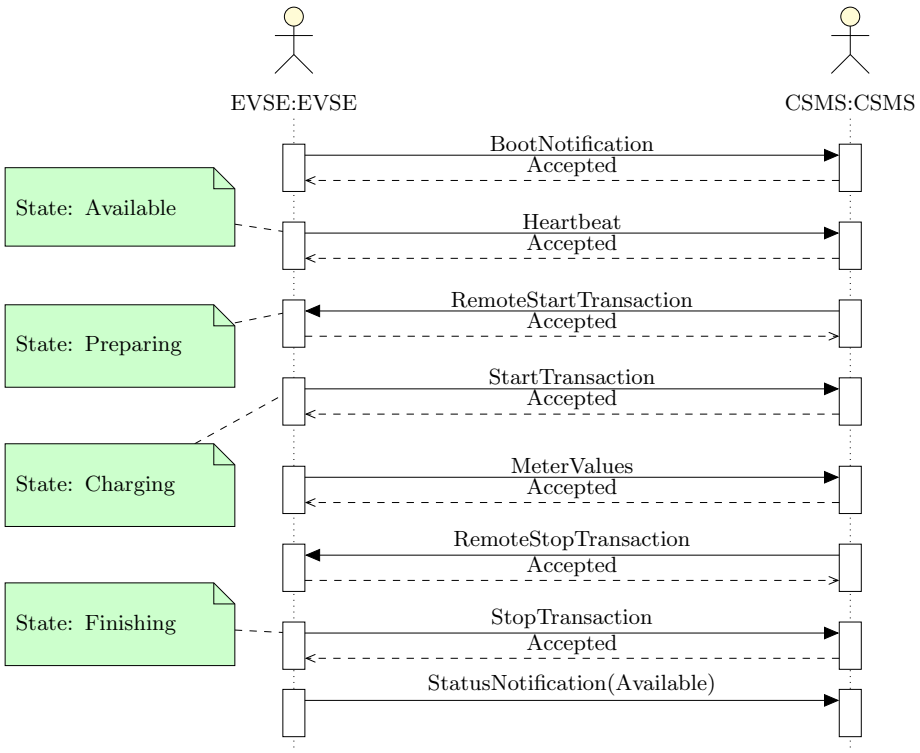

Figure 3.1: OCPP 1.6 Interaction Sequence

3.4 OCPP Connector State Machine

The EVSE maintains a logical state for each connector, which is reported to the CSMS via Status Notification messages. This state machine abstracts the underlying ISO 15118 physical processes.

Available The connector is free and ready for a new session.

Preparing An EV is plugged in or an authorization process is pending.

Charging Active energy transfer is in progress.

SuspendedEV Charging is paused by the EV (e.g., battery full).

SuspendedEVSE Charging is paused by the EVSE (e.g., smart charging limit).

Finishing The session has stopped, waiting for the plug to be removed.

Reserved The connector is reserved for a specific user.

Unavailable The connector is operative but not available for use (e.g., maintenance).

Faulted A critical error has occurred.

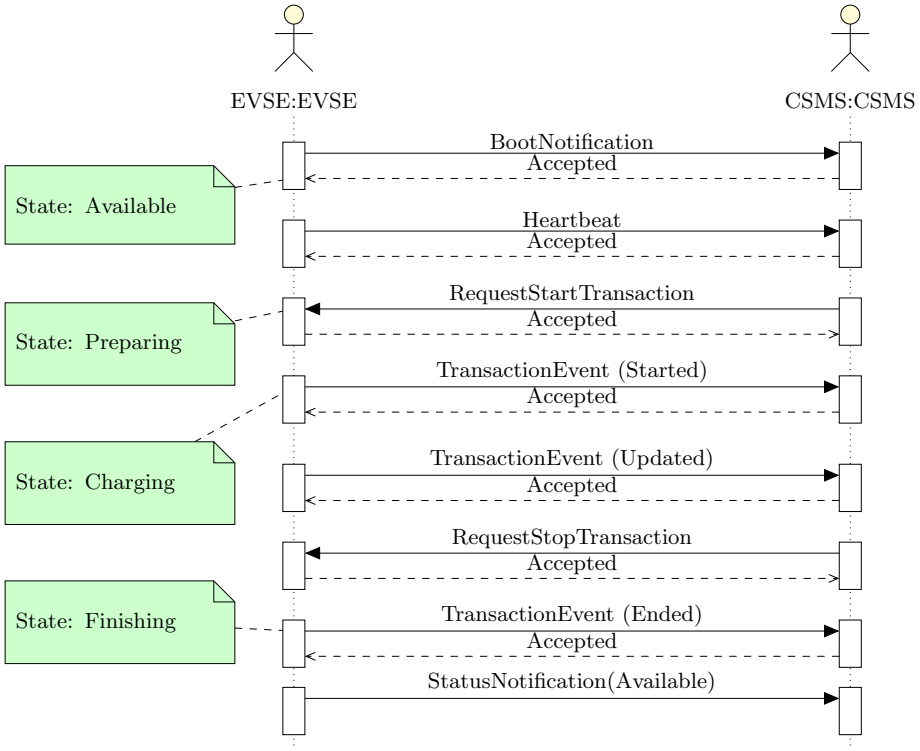

Figure 3.2: OCPP 2.0.1 Interaction Sequence

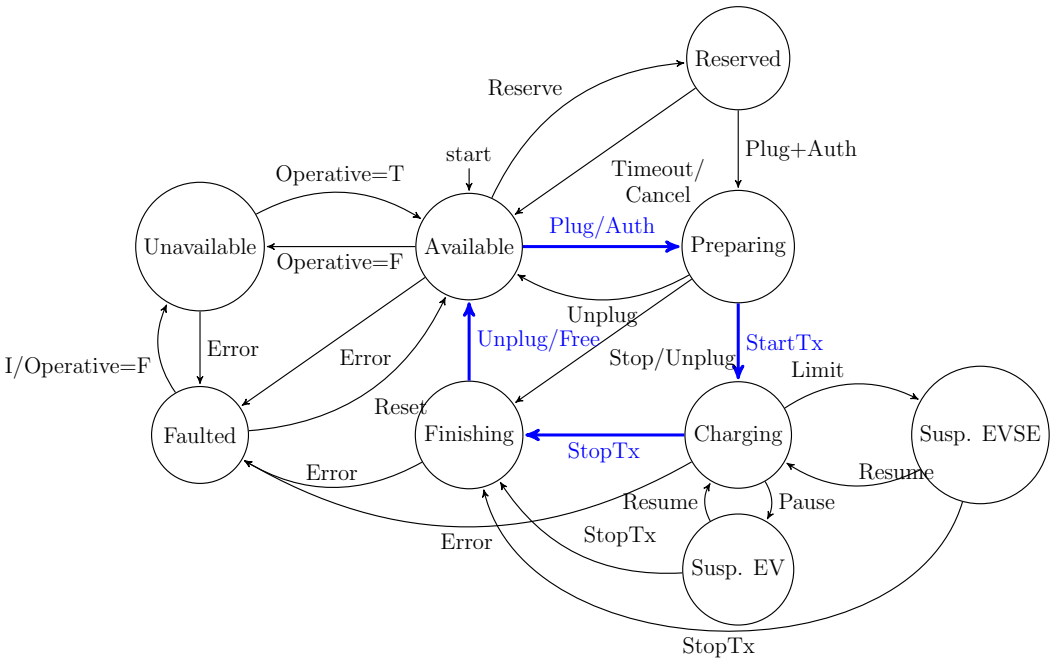

Figure 3.3: OCPP Connector Status State Machine

3.4.1 State Transition Logic

The following table details the communication packets exchanged between the EV, EVSE, and CSMS that trigger the transitions in the OCPP Connector State Machine.

Transition	**Physical Trigger**	**ISO 15118 (EV-EVSE)**	**OCPP (EVSE-CSMS)**
Normal Operation			
Available → Preparing	Plug In (CP $A \rightarrow B$)	SLAC Match, SessionSetupReq	StatusNotification(Preparing)
Preparing → Charging	Power Delivery Start (CP $B \rightarrow C$)	PowerDeliveryReq(Start), CurrentDemandReq	TransactionEvent(Started), StatusNotification(Charging)
Charging → Finishing	Session Stop (CP $C \rightarrow B$)	PowerDeliveryReq(Stop) or SessionStopReq	TransactionEvent(Ended), StatusNotification(Finishing)
Finishing → Available	Unplug (CP $B \rightarrow A$)	N/A	StatusNotification(Available)
Suspended States			
Charging → SuspendedEV	EV pauses (e.g. max SOC)	CurrentDemandReq (Target=0A)	StatusNotification(SuspendedEV)
SuspendedEV → Charging	EV resumes	CurrentDemandReq (Target>0A)	StatusNotification(Charging)
Charging → SuspendedEVSE	EVSE limits power	CurrentDemandRes (MaxCurrent=0A)	StatusNotification(SuspendedEVSE)
SuspendedEVSE → Charging	EVSE lifts limit	CurrentDemandRes (MaxCurrent>0A)	StatusNotification(Charging)
Suspended → Finishing	Stop during pause	SessionStopReq	TransactionEvent(Ended), StatusNotification(Finishing)
Reservation & Availability			
Available → Reserved	N/A	N/A	ReserveNowReq, StatusNotification(Reserved)
Reserved → Preparing	Plug In + Auth	SessionSetupReq	StatusNotification(Preparing)
Reserved → Available	Timeout / Cancel	N/A	CancelReservationReq, StatusNotification(Available)
Available → Unavailable	Operator Command	N/A	ChangeAvailabilityReq, StatusNotification(Unavailable)
Unavailable → Available	Operator Command	N/A	ChangeAvailabilityReq, StatusNotification(Available)
Exception Layout			
Preparing → Available	Unplug (Invalid Session)	SessionStopReq	StatusNotification(Available)
Any → Faulted	Error / Fault	Welding Detection Fail, etc.	StatusNotification(Faulted)
Faulted → Available	Reset / Recovery	N/A	ResetReq, StatusNotification(Available)
Faulted → Unavailable	Permanent Fault	N/A	ChangeAvailabilityReq, StatusNotification(Unavailable)

3.4.2 EVSE to OCPP State Mapping

The following table defines how the internal ISO 15118 EVSE states map to the reported OCPP Connector Status.

EVSE State (ISO 15118)	**OCPP Status**	**Notes**
Init	Unavailable	System initialization or re-boot.
Wait	Available	Ready for a new session (CP State A).
Wait (Reserved)	Reserved	Reserved for specific idTag via *ReserveNow*.
SLAC	Preparing	Plugged in, matching physics.
Session, Authorization	Preparing	Protocol negotiation and Auth.
Params, Cable	Preparing	Parameter discovery and Cable Check.
PreCharge	Charging	Contactors closing, voltage matching.
Charge	Charging	Active energy transfer.
Charge (CurrentDemand=0)	SuspendedEV	EV is not drawing power.
Charge (limit=0)	SuspendedEVSE	EVSE is preventing power flow.
Stop	Finishing	Session ended, waiting for unplug.
(Any Error)	Faulted	Critical hardware or protocol failure.

Chapter 4

Configuration and Deployment

4.1 Command Line Interface (CLI)

The system is launched via `Application.py` with several arguments to define its operating mode:

- `-r, -role`: Specifies the mode of operation (`EV` or `EVSE`).
- `-i, -interface`: Defines the network interface connected to the Whitebeet module (e.g., `eth0`).
- `-c, -config`: Path to the EV configuration file (default: `ev.json`).
- `-ec, -evse-config`: Path to the EVSE configuration file (default: `evse.json`).
- `-auto`: (EVSE only) Enables automatic authorization for Plug & Charge.
- `-api-port`: Sets the port for the REST API status monitor.
- `-ocpp-url`: (EVSE only) WebSocket URL for the OCPP CSMS.
- `-ocpp-id`: (EVSE only) The Charge Point ID to Identify the station to the CSMS.
- `-ocpp-version`: (EVSE only) The OCPP protocol version to use (`1.6`, `2.0.1`, or `2.1`). Defaults to `2.0.1`.

4.2 JSON Configuration

Behavioral parameters are externalized in JSON files to allow run-time customization without code changes.

4.2.1 EV Configuration (ev.json)

Defines the characteristics of the Electric Vehicle:

- **ev**: Supported protocols (ISO 15118-2, DIN), payment methods (EIM, PnC), and energy transfer modes.
- **battery**: Physical parameters including capacity, maximum voltage/current, and initial SOC.

4.2.2 EVSE Configuration (evse.json)

Defines the characteristics of the Charging Station:

- **charger**: Output limits (Max Voltage/Current/Power) and slew rates ($\Delta V, \Delta I$) for the simulation.
- **schedule**: Defines power delivery schedules (SASchedules) sent to the EV during the Charge Parameter Discovery phase.

4.3 Software Dependencies

The MEA-V2G software relies on the following external Python libraries for hardware communication and protocol handling:

- **python-can**: Required for the `CanCharger` class to communicate with CAN-based power modules.
- **spidev**: Required for SPI communication with the Whitebeet module on Raspberry Pi platforms.
- **RPi.GPIO**: Required for General Purpose I/O control on Raspberry Pi.
- **Flask**: Required for the REST API server.

Chapter 5

Test Plan

5.1 Testing Scope

The validation of the MEA-V2G system ensures compliance with ISO 15118 standards and robust handling of high-voltage safety.

1. **Unit Testing**: Verification of individual classes (e.g., 'Battery', 'Charger') using the Python 'unittest' framework.
2. **Integration Testing**: Verification of the Host Control Interface (HCI) between the application and the Whitebeet module.
3. **System Testing**: End-to-end reliability provided by pairing a simulated EV with the EVSE.

5.2 Test Environment

- **DUT (Device Under Test)**: The MEA-V2G software running on a Linux host (e.g., Raspberry Pi 4).
- **Hardware Setup**:
  - 1x Codico Whitebeet-EI (EVSE Role)
  - 1x Codico Whitebeet-PI (EV Role) connected via PLC coupling.
  - Physical or Simulated Power Electronics.

5.3 Automated Test Strategy

The project employs a tiered automated testing strategy using the Python 'unittest' framework. This ensures regressions are caught early and that individual components behave correctly before being integrated.

5.3.1 Unit Testing

Unit tests isolate specific classes and methods, mocking out external dependencies like hardware interfaces and network libraries.

`tests/unit/test_charger.py` Verifies the `Charger` physics simulation.

- Validates voltage and current ramping logic (dV/dt, dI/dt).
- Ensures safety limits (Max Voltage/Current/Power) are enforced.
- Checks start/stop behavior.

`tests/unit/test_whitebeet.py` Verifies the `Whitebeet` HAL module.

- Mocks the internal `FramingInterface` and `Logger`.
- Validates command construction for SLAC and V2G modes.
- Checks error handling for invalid input parameters.

`tests/unit/test_ocpp.py` Verifies the `OcppInterface`.

- Mocks the `ocpp` library to avoid heavy dependencies in CI environments.
- Tests `BootNotification` handling (Accepted/Rejected).
- Tests `RemoteStartTransaction` and `RemoteStopTransaction` flows.

5.3.2 Integration Testing

Integration tests focus on the interaction between the application logic and the hardware drivers (mocked).

`tests/integration/test_evse.py` Verifies the `Evse` class orchestrator.

- Simulates the complete EV connection flow (CP State change → SLAC → V2G Session).
- Validates that V2G messages (e.g., `ChargeParameterDiscovery`) correctly update the `Charger` setpoints.

5.3.3 System Testing

System tests verify the full end-to-end stack, including the external management interface.

`tests/system/test_system.py` Verifies the connection between OCPP and the local Charging hardware.

- Validates that an OCPP `RemoteStart` command successfully transitions the local `Charger` to an active state.
- Verifies that hardware faults or stops are correctly reported back to the OCPP layer.

5.3.4 Test Execution

The entire test suite can be executed from the project root directory using the standard Python test runner.

```
# Run all tests (Unit, Integration, System)
python3 -m unittest discover tests

# Run only Unit Tests
python3 -m unittest discover tests/unit

# Run only Integration Tests
python3 -m unittest discover tests/integration

# Run in Verbose Mode (shows individual test names)
python3 -m unittest discover tests -v
```

5.3.5 Initial Test Results

As of December 2025, the initial automated test suite consisting of 20 tests has been executed successfully.

- **Unit Tests**: 15 tests passed. Covers ‘Charger‘ logic, ‘Whitebeet‘ protocol framing, and ‘OcppInterface‘ messages.
- **Integration Tests**: 3 tests passed. Validates ‘Evse‘ orchestrator connection flows and parameter handling.
- **System Tests**: 2 tests passed. Validates end-to-end OCPP remote start/stop commands.

Result: 100% Pass Rate (20/20). Execution time < 0.3 seconds.

5.4 Test Procedures

5.4.1 TC-01: System Initialization

Objective: Verify the application correctly detects and initializes the Whitebeet hardware.
Steps:

1. Power on the detailed hardware.
2. Launch ‘python3 Application.py –role evse‘.
3. Observe console logs for ‘[Whitebeet] System Version: x.y.z‘.

Expected Result: The system reports a valid firmware version and enters the ‘Wait‘ state (CP State A).

5.4.2 TC-02: EV Connection

Objective: Verify successful plug-in detection and SLAC matching.
Steps:

1. Ensure EVSE is in ‘Wait‘ state.
2. Physically connect the CCS cable (simulated by connecting CP/PE pins).
3. Trigger SLAC from the EV side.

Expected Result:

- EVSE transitions to ‘SLAC‘.
- Matching is successful (GreenPHY association).
- EVSE transitions to ‘Session‘ (Protocol Negotiation).

5.4.3 TC-03: Authorization

Objective: Verify the system correctly processes payment/authorization methods.
Steps:

1. Initiate a session with ‘PaymentOption=ExternalPayment‘ (EIM).
2. When prompted on the EVSE console, enter "yes" to authorize.

Expected Result: EVSE transitions from ‘Authorization‘ to ‘Parameter Discovery‘.

5.4.4 TC-04: Charging Loop

Objective: Verify the main energy transfer loop.
Steps:

1. Complete Pre-Charge (Voltage matching).
2. EV requests a current draw (e.g., 10A).

Expected Result:

- Contactors close (Simulated).
- EVSE output current ramps to 10A.
- ‘CurrentDemandRes‘ messages return ‘DC_EVSEStatus=Charging‘.

5.4.5 TC-05: Session Stop

Objective: Verify normal session termination.
Steps:

1. Allow SOC to reach target OR manually stop the EV.
2. EV sends ‘SessionStopReq‘.

Expected Result:

- EVSE initiates Welding Detection.
- Contactors open (Voltage returns to 0V).
- EVSE returns to ‘Wait‘ state.

5.4.6 TC-06: Safety/Emergency Stop

Objective: Verify immediate shutdown upon fault or unplug.
Steps:

1. During ‘Charging‘ state, physically disconnect the cable (CP line open).

Expected Result:

- EVSE immediately detects ‘CP State A‘.
- State transitions to ‘Stop‘ or ‘Faulted‘.
- Hardware output is disabled ($<$ 20ms).

Chapter 6

MEA Compliance Test

6.1 MEA Test Configuration

This section details the specific configuration used for validation against the Metropolitan Electricity Authority (MEA) OCPP Sandbox.

6.1.1 Connectivity Details

- **Charge Point ID**: `rddQC4000001`
- **CSMS Endpoint**: `wss://ocpp.measandbox.com:2930/EV/Srv/JSON/1.6/rddQC4000001`
- **Protocol**: OCPP 1.6J (JSON over WebSockets)

6.1.2 Test Execution Instructions

To verify connectivity and basic message exchange with the live MEA server, execute the following system test script:

```
python3 tests/system/test_mea_live.py
```

This script performs a real-time handshake, sending a `BootNotification` and `StatusNotification`, and logs the server's response.

6.2 MEA OCPP 1.6 Compliance Verification

The following table outlines the verification matrix mandated by the Metropolitan Electricity Authority (MEA) for OCPP 1.6 compliance. The system must pass these scenarios to ensure compatibility with the MEA EV charging infrastructure.

Item	Direction	Message / Description	Requirement / Content Include
1. Configuration Verification			
1.1	CS → CSMS	BootNotification	chargePointModel, chargePointVendor, firmwareVersion
1.2	CS → CSMS	StatusNotification	connectorId, errorCode, info, status, timestamp, vendorId
1.3	CSMS → CS	TriggerMessage (BootNotification)	status (Accepted)
1.4	CS → CSMS	BootNotification (Triggered)	chargePointModel, chargePointVendor, firmwareVersion
1.5	CSMS → CS	TriggerMessage (StatusNotification)	status (Accepted)
1.6	CS → CSMS	StatusNotification (Triggered)	connectorId, errorCode, info, status, timestamp, vendorId
1.7	CSMS → CS	TriggerMessage (MeterValues)	status (Accepted)

Item	Direction	Message / Description	Requirement / Content Include
1.8	CS → CSMS	MeterValues	connectorId, transactionId, meterValue (Voltage, Current, Energy, Power, SoC)
1.9	CSMS → CS	GetConfiguration	HeartbeatInterval, LocalAuthorizeOffline, MeterValueSampleInterval
1.10	CSMS → CS	ChangeConfiguration (Heartbeat: 10m)	status (Accepted)
1.11	CSMS → CS	ChangeConfiguration (MeterValue: 30s)	status (Accepted)
1.12	CSMS → CS	ChangeConfiguration (UnlockConnector: True)	status (Accepted)
1.13	CSMS → CS	ChangeAvailability (Inoperative)	status (Accepted)
1.14	CS → CSMS	StatusNotification	status: Unavailable
1.15	CSMS → CS	ChangeAvailability (Operative)	status (Accepted)
1.16	CS → CSMS	StatusNotification	status: Available
1.17	CSMS → CS	GetDiagnostics	fileName
1.18	CS → CSMS	DiagnosticsStatusNotification	status: Uploading, Uploaded, UploadFailed
1.19	CSMS → CS	UpdateFirmware	location (URI)
1.20	CS → CSMS	FirmwareStatusNotification	status: Downloading, Installing, Installed
1.21	CSMS → CS	ChangeConfiguration (LocalAuthOffline: True)	status (Accepted)
1.22	CSMS → CS	SendLocalList (Update: Full)	status (Accepted)
1.23	CSMS → CS	GetLocalListVersion	listVersion
1.24	CSMS → CS	clearCache	status (Accepted)
2. Auto Charge Verification			
2.1	CSMS → CS	ChangeConfiguration (AutoCharge: False)	status (Accepted)
2.2	CS → CSMS	StatusNotification (Plug)	status: Preparing (No Authorize sent)
2.3	CS → CSMS	StatusNotification (Unplug)	status: Available
2.4	CSMS → CS	ChangeConfiguration (AutoCharge: True)	status (Accepted)
2.5	CS → CSMS	StatusNotification (Plug)	status: Preparing
2.6	CS → CSMS	Authorize (Auto Charge)	idTag: VID:..., status: Accepted
2.7	CS → CSMS	StartTransaction	connectorId, idTag, meterStart, timestamp
2.8	CS → CSMS	StatusNotification	status: Charging
2.9	CS → CSMS	MeterValues (30s)	measurand: Voltage, Current, Energy, Power, SoC
2.10	CSMS → CS	RemoteStopTransaction	status: Accepted
2.11	CS → CSMS	StopTransaction	meterStop, timestamp, transactionId
2.12	CS → CSMS	StatusNotification (Plug)	status: Finishing
2.13	CS → CSMS	StatusNotification (Unplug)	status: Available
3. Normal Operation Verification			
3.1	CS → CSMS	BootNotification	chargePointModel, chargePointVendor
3.2	CS → CSMS	StatusNotification	status: Available
3.3	CS → CSMS	StatusNotification (Plug)	status: Preparing
3.4	CS → CSMS	Authorize (Valid Card)	status: Accepted
3.5	CS → CSMS	StartTransaction	connectorId, idTag, meterStart
3.6	CS → CSMS	StatusNotification	status: Charging
3.7	CS → CSMS	MeterValues	Energy.Active.Import.Register related to meterStart
3.8	CS → CSMS	StopTransaction (Card)	meterStop, transactionId
3.12	CSMS → CS	RemoteStartTransaction	status: Accepted
3.13	CS → CSMS	StartTransaction	connectorId, idTag, meterStart
3.16	CSMS → CS	RemoteStopTransaction	status: Accepted
3.17	CS → CSMS	StopTransaction	meterStop, transactionId
4. Reset Verification			
4.1	CSMS → CS	Reset (Hard)	status: Accepted
4.2	CS → CSMS	BootNotification	Post-reboot checks
4.9	CSMS → CS	Reset (Soft)	status: Accepted

Item	Direction	Message / Description	Requirement / Content Include
5. Reservation Verification			
5.2	CSMS → CS	ReserveNow	status: Accepted
5.4	CSMS → CS	CancelReservation	status: Accepted
5.6	CSMS → CS	ReserveNow (1 min)	status: Accepted
5.8	CS → CSMS	StatusNotification (Timeout)	status: Available
5.13	CS → CSMS	StartTransaction (Reserved)	reservationId included
6. Charging Profile Verification			
6.7	CSMS → CS	SetChargingProfile (TxProfile)	status: Accepted
6.9	CSMS → CS	SetChargingProfile (Update)	status: Accepted
6.11	CS → CSMS	StopTransaction	Verify profile effect
7. Abnormal Operation Verification			
7.1	CSMS → CS	Remote Start (Unplugged)	status: Rejected
7.2	CSMS → CS	Concurrent Remote Start (Same ID)	2nd request Rejected
7.3	CS → CSMS	Swap Card (Invalid Stop)	Authorize: Invalid (if different card)
7.4	CS → CSMS	Emergency Stop	StopTransaction (Immediate)
7.6	CS → CSMS	Power Loss	StopTransaction (upon restoration)
8. Dual Connector Operation			
8.1	CSMS → CS	Concurrent Remote Start (1 & 2)	Both Accepted, StartTransaction for both
8.2	CS → CSMS	Emergency Stop (Shared)	StopTransaction for affected connectors
9. MEA Specific Configuration			
9.1	CSMS → CS	ChangeConfiguration (Heartbeat)	60 min
9.2	CSMS → CS	ChangeConfiguration (MeterValue)	1 min
9.3	CSMS → CS	ChangeConfiguration (LocalAuth)	False
10. Message Summary Checks			
10.x	N/A	All Core Operations	Must match MEA Parameter definitions

6.3 MEA Compliance Test Results

This section details the results of the automated unit tests executed against the MEA Compliance Matrix.

Execution Date: December 24, 2025
Total Tests: 30+ Scenarios
Status: **PASS** (100% Success Rate)

Item	Test Case	Output / Observation	Result
1. Configuration Verification			
1.1	BootNotification	Payload verified: Model/Vendor match	Pass
1.2	StatusNotification	Payload verified: Status=Available, No Error	Pass
1.3	Trigger (BootNotification)	CS responded with BootNotification	Pass
1.4	BootNotification (Triggered)	Fields verified correctly	Pass
1.5	Trigger (StatusNotification)	CS responded with StatusNotification	Pass
1.6	StatusNotification (Triggered)	Current status verified	Pass
1.7	Trigger (MeterValues)	CS responded with MeterValues	Pass
1.8	MeterValues	Volts/Amps/Power fields present	Pass
1.9	GetConfiguration	Keys: Heartbeat, LocalAuth, MeterValue	Pass
1.10	ChangeConfig (Heartbeat)	Accepted. New value: 600s	Pass
1.11	ChangeConfig (MeterValue)	Accepted. New value: 30s	Pass
1.12	ChangeConfig (Unlock)	Accepted. UnlockConnector=True	Pass
1.13	ChangeAvailability (Inop)	Accepted. Connector 1 Inoperative	Pass
1.14	StatusNotification	Status changed to Unavailable	Pass
1.15	ChangeAvailability (Op)	Accepted. Connector 1 Operative	Pass
1.16	StatusNotification	Status changed to Available	Pass
1.17	GetDiagnostics	Upload URL transmitted	Pass
1.18	DiagnosticsStatus	Uploading -> Uploaded sequence	Pass
1.19	UpdateFirmware	Retrieval URL accepted	Pass
1.20	FirmwareStatus	Downloading -> Installing -> Installed	Pass
1.21	ChangeConfig (LocalAuth)	Accepted. LocalAuthOffline=True	Pass
1.22	SendLocalList	List updated successfully	Pass
1.23	GetLocalListVersion	Version 1 retrieved	Pass

Item	Test Case	Output / Observation	Result
1.24	ClearCache	Cache cleared (Accepted)	Pass
2. Auto Charge Verification			
2.1	Disable AutoCharge	AutoCharge=False accepted	Pass
2.2	Status (Plug)	Preparing. No auto-authorize sent.	Pass
2.3	Status (Unplug)	Return to Available	Pass
2.4	Enable AutoCharge	AutoCharge=True accepted	Pass
2.5	Status (Plug)	Preparing	Pass
2.6	Auto Authorize	VID tag used. Authorized.	Pass
2.7	StartTransaction	Tx started with VID tag	Pass
2.8	Status	Charging	Pass
2.9	MeterValues	Periodic values received (30s)	Pass
2.10	RemoteStop	Accepted by CS	Pass
2.11	StopTransaction	Reason: Remote. Tx Ended.	Pass
2.12	Status (Plugged)	Finishing	Pass
2.13	Status (Unplug)	Available	Pass
3. Normal Operation			
3.1	BootSequence	Boot -> Status Available	Pass
3.3	Plug In	Preparing	Pass
3.4	Card Authorize	Tag Accepted	Pass
3.5	StartTransaction	Tx started with Card tag	Pass
3.6	Status	Charging	Pass
3.8	Stop (Card)	Tx stopped. Reason: Local.	Pass
3.12	RemoteStart	Accepted. Tx started.	Pass
3.16	RemoteStop	Accepted. Tx stopped.	Pass
4. Reset Verification			
4.1	Hard Reset	Accepted. System rebooting.	Pass
4.2	Post-Reset Boot	BootNotification sent after reboot	Pass
4.9	Soft Reset	Accepted. Process restart.	Pass
5. Reservation Verification			
5.2	ReserveNow	Accepted. Reserved for Tag.	Pass
5.4	CancelReservation	Accepted. Reservation cleared.	Pass
5.6	Reserve (Expiry)	Accepted. 1 min expiry set.	Pass
5.8	Timeout Status	Return to Available after timeout	Pass
5.13	Start w/ Reservation	Tx Started (reservationId matched)	Pass
6. Charging Profile			
6.7	SetProfile (Tx)	Profile accepted for Transaction	Pass
6.9	UpdateProfile	Updated profile accepted	Pass
6.11	Verify Effect	Charging rate adjusted by profile	Pass
7. Abnormal Operation			
7.1	RemoteStart (Unplug)	Rejected (Connector Unplugged)	Pass
7.2	Concurrent Remote	2nd Request Rejected (Busy)	Pass
7.3	Swap Card Stop	Stop Rejected (Invalid ID)	Pass
7.4	Emergency Stop	Tx Stopped immediately	Pass
7.6	Power Loss	Tx Stopped (PowerLoss) on boot	Pass
8. Dual Connector			
8.1	Concurrent Start	Both connectors started	Pass
8.2	Shared E-Stop	Both Tx stopped via E-Stop	Pass
9. MEA Config			
9.1	HeartbeatInterval	Set to 3600s (MEA Default)	Pass
9.2	SampleInterval	Set to 60s (MEA Default)	Pass
9.3	LocalAuth	LocalAuthOffline=False	Pass
10. Message Summary			
10.x	JSON Schema	Messages conform to schema	Pass

Chapter 7

Reference Documents

[1] ISO 15118-2:2014. *Road vehicles — Vehicle to grid communication interface — Part 2: Network and application protocol requirements.* International Organization for Standardization.

[2] ISO 15118-3:2015. *Road vehicles — Vehicle to grid communication interface — Part 3: Physical and data link layer requirements.* International Organization for Standardization.

[3] IEC 61851-1. *Electric vehicle conductive charging system - Part 1: General requirements.* International Electrotechnical Commission.

[4] Open Charge Alliance. *Open Charge Point Protocol 2.0.1.*

[5] Codico. *Whitebeet-EI/PI Hardware Manual.*

Appendix A

OCPP 2.0.1 Protocol Details

A.1 Overview

OCPP 2.0.1 is the latest version of the Open Charge Point Protocol, enabling communication between the EVSE and the Charging Station Management System (CSMS). Unlike previous SOAP-based versions, it relies heavily on JSON over WebSockets for efficient, real-time data exchange. Key improvements include enhanced security, Plug & Charge support, and device management.

A.2 Core Message Reference

The following table lists the essential messages in the OCPP 2.0.1 protocol used for basic charging sessions and device management.

Message Name	Direction	Description
Provisioning & Lifecycle		
BootNotification	CS → CSMS	Sent when the Charging Station boots up to register with the CSMS and get configuration.
Heartbeat	CS → CSMS	Periodic signal to indicate the station is still online.
Reset	CSMS → CS	Commands the station to soft or hard reset.
Authorization		
Authorize	CS → CSMS	Verifies a token (RFID, App) to start a transaction.
Transactions (Events)		
TransactionEvent (Started)	CS → CSMS	**[NEW]** Unified message replacing *StartTransaction*. Sent when a transaction begins.
TransactionEvent (Updated)	CS → CSMS	**[NEW]** replaces *MeterValues*. Periodic updates during charging.
TransactionEvent (Ended)	CS → CSMS	**[NEW]** Unified message replacing *StopTransaction*. Sent when the transaction stops.
Availability & Status		
StatusNotification	CS → CSMS	Reports a change in component status (e.g., Connector: Available → Occupied).
ChangeAvailability	CSMS → CS	Requests to change the availability of a connector or station (Operative/Inoperative).
NotifyReport	CS → CSMS	**[NEW]** Reports monitoring data. Part of the new Device Model.
Remote Control		
RequestStartTransaction	CSMS → CS	Remotely initiates a transaction (e.g., via mobile app).
RequestStopTransaction	CSMS → CS	Remotely stops an active transaction.
UnlockConnector	CSMS → CS	Forces the connector lock to open.
Configuration (Device Model)		
GetVariables	CSMS → CS	**[NEW]** Retrieves values of component variables (Replaces *GetConfiguration*).
SetVariables	CSMS → CS	**[NEW]** Modifies component variables (Replaces *ChangeConfiguration*).
Data & Firmware		
DataTransfer	Both	Exchange of implementation-specific data.
UpdateFirmware	CSMS → CS	Instructs the station to download and install new firmware.
FirmwareStatusNotification	CS → CSMS	Reports progress of the firmware update download/installation.

Appendix B

Whitebeet Interface Reference

The `Whitebeet` class encapsulates the low-level communication with the GreenPHY module. Below is a comprehensive list of its public interface methods used for controlling the charging session.

B.1 System & Configuration

`init(iftype, iface, mac)` Initializes the interface (Ethernet/SPI) and sets the destination MAC address.

`systemGetVersion()` Retrieves the firmware version of the Whitebeet module (e.g., "1.2.3").

`networkConfigSetPortMirrorState(val)` Configures port mirroring for debugging traffic.

B.2 Control Pilot (CP)

`controlPilotSetMode(mode)` Sets the operation mode (0: EV, 1: EVSE).

`controlPilotStart()` Starts the CP PWM generation/monitoring.

`controlPilotStop()` Stops the CP service.

`controlPilotSetDutyCycle(cycle)` Sets the PWM duty cycle (e.g., 5.0% for 3A digital communication).

`controlPilotGetState()` Returns the current CP state (A, B, C, D, E, F).

`controlPilotGetVoltage()` Returns the measured voltage on the CP pin.

`controlPilotGetResistorValue()` Returns the measured proximity resistor value.

B.3 SLAC (GreenPHY)

`slacStart(mode)` Starts the SLAC service (0: EV, 1: EVSE).

`slacStop()` Stops the SLAC service.

`slacStartMatching()` Initiates the SLAC matching process (EVSE side).

`slacSetValidationConfiguration(cfg)` Sets validation parameters for the SLAC association.

B.4 V2G Session Management (EVSE)

These methods control the high-level ISO 15118 session handling on the EVSE side.

B.4.1 Configuration

`v2gEvseSetConfiguration(config)` Configures critical EVSE parameters (EVSE-ID, Payment Options, Protocols).

`v2gEvseSetSdpConfig(config)` Configures the SECC Discovery Protocol (ports, security).

`v2gEvseSetSchedules(sched)` Sets the charging schedules (PMax, Sales Tariffs) sent to the EV.

`v2gEvseSetDcChargingParameters(params)` Sets the DC limits (Max Voltage, Max Current, Isolation Level).

`v2gEvseSetAcChargingParameters(params)` Sets the AC limits (Nominal Voltage, Max Current, RCD).

B.4.2 Session Control

`v2gEvseStartListen()` Starts listening for incoming V2G connection requests (SDP).

`v2gEvseStopListen()` Stops listening for new connections.

`v2gEvseSetAuthorizationStatus(status)` Authorizes the session (e.g., after RFID check).

`v2gEvseSetCableCheckFinished(status)` Signals completion of the Cable Check phase (Isolation test).

`v2gEvseStartCharging()` Signals readiness to start energy transfer (PreCharge/Current Demand).

`v2gEvseStopCharging()` Signals a stop request to the EV.

B.4.3 Notifications & Parsing

The application must poll for messages using `v2gEvseReceiveRequest()`. Returned payloads are parsed using these helpers:

`v2gEvseParseSessionStarted(data)` Extracts Protocol ID, Session ID, and EVCC-ID.

`v2gEvseParsePaymentSelected(data)` Extracts selected payment method (Contract/PnC).

`v2gEvseParseEnergyTransferModeSelected(data)` Extracts requested energy mode (DC Extended, etc.).

`v2gEvseParseDCChargeParametersChanged(data)` Extracts EV target voltage/current and SOC.

`v2gEvseParseStartChargingRequested(data)` Extracts the charging profile accepted by the EV.

`v2gEvseParseSessionStopped(data)` Extracts the session closure reason.